

What is React?

React is an open-source JavaScript library for building user interfaces, especially interactive web applications. It lets you create reusable UI pieces called components, combine them into pages, and update only the parts of the screen that change.

basic function

```
function Welcome() {  
  
  return
```

Hello, Subsu!

```
;  
  
}
```

Components: Reusable building blocks such as buttons, navigation bars, forms, and cards.

JSX: A syntax that allows you to write UI markup inside JavaScript.

Props: Data passed from one component to another.

State: Data that can change, such as a counter value or login status.

Declarative UI: You describe what the screen should look like for a given state; React handles the updates.

Virtual DOM: React compares the expected UI with the previous version and efficiently updates the browser's DOM.

React is technically a library, not a complete framework.

2.Is React a library or framework?

React is a JavaScript library, not a complete framework

React mainly provides tools for:

- Creating reusable UI components.
- Managing component state.
- Handling user interactions.
- Rendering interface updates efficiently.

React is a UI library; Next.js and similar tools are frameworks that use React.

3.What is a component?

A component in React is a reusable, independent piece of a user interface. It is usually a JavaScript function that receives data called props and returns JSX describing what should appear on the screen.

Components allow you to divide a large interface into smaller parts, such as:

- Navbar
- LoginForm
- ProductCard
- UserProfile
- Footer

Each part can be developed, tested, reused, and updated independently.

4.Why are components important?

Components are important because they break a complex user interface into small, reusable, and manageable pieces. This makes React applications easier to build, understand, test, maintain, and expand.

1. **Reusability**- You can write a component once and use it in many places.
 2. **Easier maintenance**- Suppose the same button appears on ten pages. If the button is a reusable component, you can change its style or behavior in one place, and the update can apply everywhere it is used.
 3. **Less complexity** - Instead of keeping an entire application in one large file, you can divide it into focused parts.
 4. **Consistency**- Using the same Button, Input, or Card component throughout an application helps keep the design and behavior consistent. For example, every button can have the same color, spacing, hover effect, and accessibility behavior.
 5. **Simpler testing and debugging**- A small component can be tested independently. If a login form has a problem, you can inspect the LoginForm component instead of examining the entire application.
 6. **Better teamwork**- Different developers can work on different components at the same time. One developer might build the navigation bar while another works on the user profile.
 7. **Scalability** - As an application grows, components provide a structure for adding new features without making the whole codebase difficult to manage. Components can combine with other components to form increasingly complex interfaces.
-

5. What is JSX?

JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside JavaScript files. React components commonly use JSX to describe the interface they should display.

- Return one root element or a fragment.

- Close every tag.
 - Use `className` instead of `class`.
 - Use `camelCase` for many attributes, such as `onClick`.
 - JSX and React are separate concepts: JSX is a syntax extension, while React is the UI library that uses it.
-

6. What is the Real DOM?

The Real DOM, usually called the DOM or Document Object Model, is the browser's in-memory representation of an HTML document. It represents the page as a tree of nodes, such as elements, attributes, and text.

The Real DOM is the actual structure used by the browser to display and interact with the web page. It is not part of the core JavaScript language; it is a browser Web API.

7. What is the Virtual DOM?

The Virtual DOM is an in-memory representation of the UI that React uses while determining what should appear on the screen.

When a React component's state or props change, React generally:

- Calls the component again.
 - Produces a new React element tree.
 - Compares it with the previous tree.
 - Determines which actual DOM changes are necessary.
 - Applies those changes to the Real DOM.
-

8. What is reconciliation?

Reconciliation is the process React uses to compare the newly

rendered UI with the previous rendered UI and determine what must change in the Real DOM.

During reconciliation, React identifies that:

- The

element still exists.

- Its structure has not changed.
- Only its text content is different.

JSX



React elements / UI description



React renders components



Reconciliation compares old and new results



React commits necessary changes



Real DOM



Browser displays the page
